

System Architecture (Living Document)

Last updated: 2026-05-06
Owners: Platform / Core API / DevOps

1) Purpose and Scope

This document defines the current system architecture for erxes private deployment, including:

- Service architecture (gateway, core API, plugin APIs, background services)
- Data layer (database, cache, queue, search)
- Deployment topology (local/dev, containerized, production pattern)
- Network topology (ports, trust boundaries, service-to-service paths)
- Dependency map (runtime and build-time)

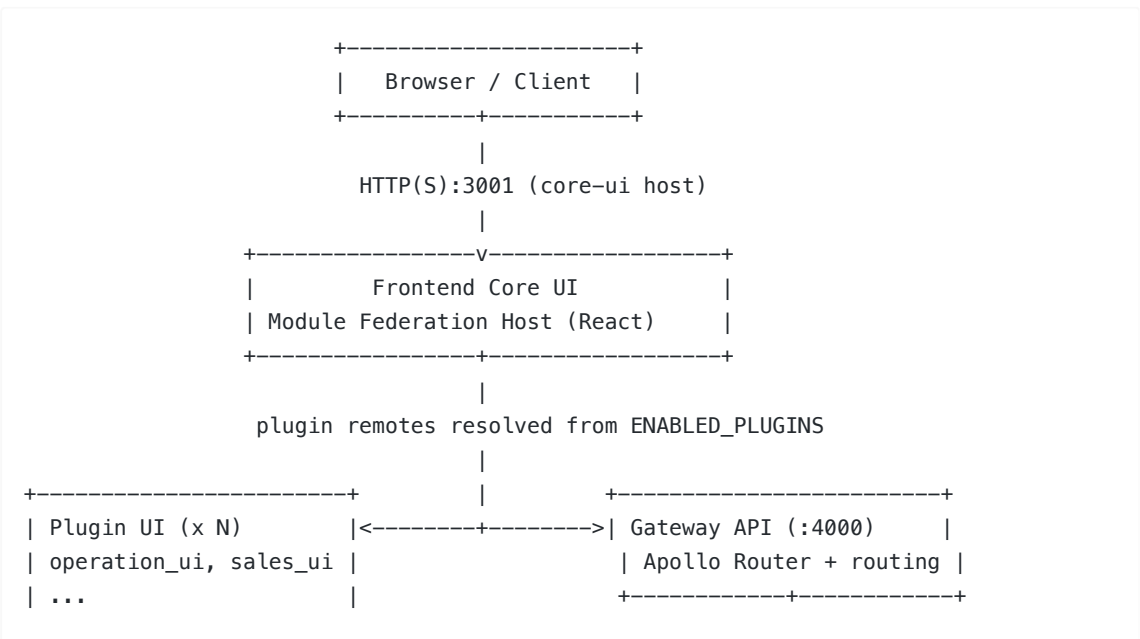
This is a living document and should be updated whenever service topology, plugin enablement, or infrastructure contracts change.

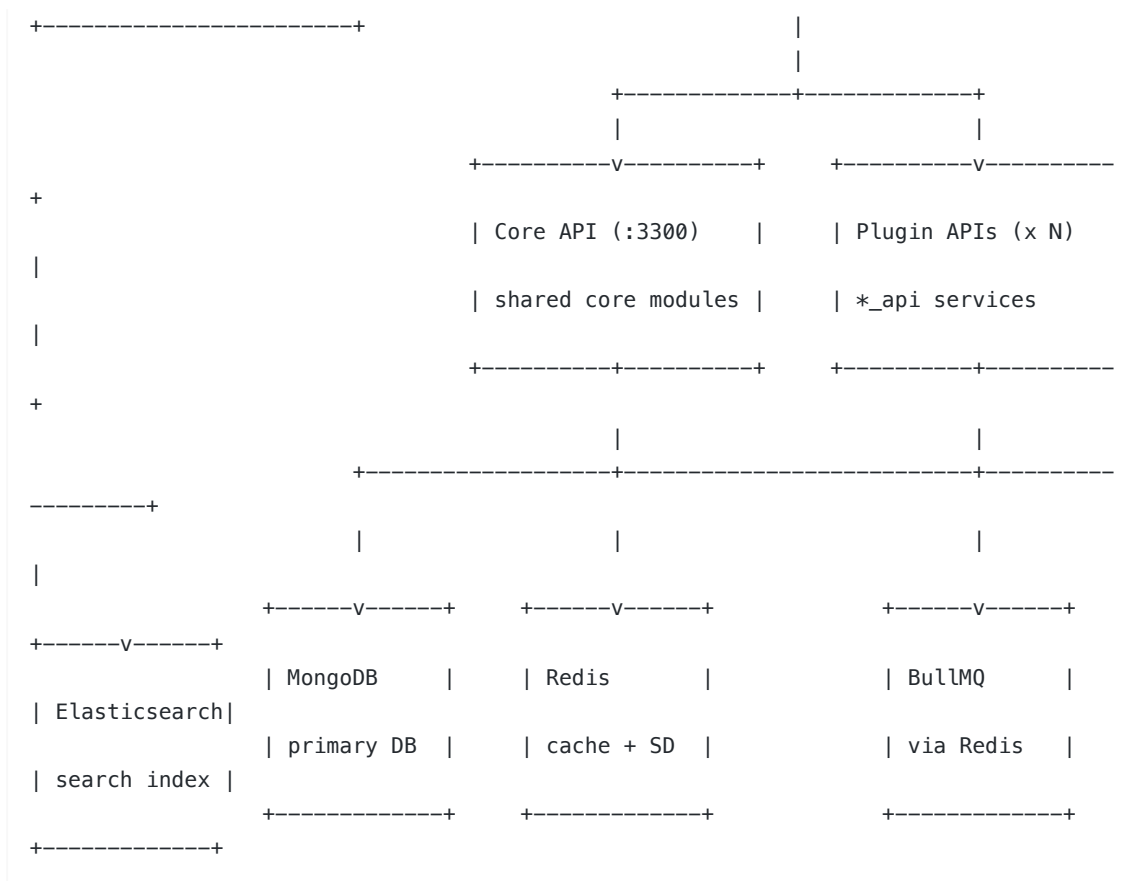
2) System Context

erxes is an Nx monorepo with microservices on the backend and module-federated micro-frontends on the frontend.

- Backend protocol surface:
 - GraphQL federation through `gateway`
 - tRPC routing through `gateway`
 - Service metadata + discovery through Redis
- Frontend protocol surface:
 - `core-ui` host application
 - Plugin UIs loaded dynamically by `ENABLED_PLUGINS`

3) High-Level Architecture





4) Services and Responsibilities

4.1 Gateway (backend/gateway)

- External backend endpoint on port `4000`
- Handles:
 - GraphQL federation routing
 - tRPC and service proxying
 - Plugin-aware routing based on runtime service discovery
- Depends on:
 - Redis for service discovery records
 - Core API and enabled plugin APIs

4.2 Core API (backend/core-api)

- Core business modules (contacts, products, segments, automations, documents, etc.)
- Typical port in development: `3300`
- Registers itself and consumes plugin availability metadata
- Depends on MongoDB, Redis, optional Elasticsearch

4.3 Plugin APIs (backend/plugins/*_api)

- Independent microservices loaded by environment configuration
- Start behavior in development is driven by:
 - `ENABLED_PLUGINS -> <plugin>_api`
 - `ENABLED_PLUGINS_ONLY_API -> API-only plugins`

- Plugins register to gateway through shared service discovery utilities
- Currently available plugin API projects in monorepo:
 - accounting , agent , block , blockadmin , blockagency , blocktest
 - btk , btkadmin , car , content , frontline , insurance , loyalty
 - mongolian , mto , mushop , onefit , operation , payment , posclient
 - sales , supplier , tourism

4.4 Background Services (backend/services)

- automations service for asynchronous business workflows
- logs service for centralized logging and processing
- Enabled in local development through ENABLED_SERVICES mapping to <name>-service

4.5 Frontend Core + Plugin UIs

- frontend/core-ui : host shell (module federation host)
- Plugin UIs loaded from ENABLED_PLUGINS as <plugin>-ui
- Core UI and plugins communicate with backend through gateway

5) Data and Messaging Architecture

5.1 MongoDB

- System of record for tenant-scoped business data
- Multi-tenancy is subdomain-aware at service context/model level
- Shared across core and plugin APIs

5.2 Redis

- Service discovery registry (service address + config metadata)
- Runtime cache
- Pub/sub and coordination
- BullMQ backing store

Key discovery records:

- erxes-service-{serviceName} -> service address
- erxesservice:config:{serviceName} -> service metadata/config

5.3 Message Queue (BullMQ)

- Queue infrastructure for delayed/retry/asynchronous jobs
- Runs over Redis
- Used by background workers and plugin automation flows

5.4 Elasticsearch (optional per feature path)

- Search indexing and retrieval for modules/features that require full-text or advanced query capabilities

6) Deployment Topology

6.1 Local Development Topology

- pnpm dev:core-api runs:
 - gateway
 - core-api

- `pnpm dev:apis` runs:
 - `core-api` + enabled plugin APIs + `gateway`
- `pnpm dev:uis` runs:
 - enabled plugin UIs
- Infra services expected:
 - MongoDB (27017)
 - Redis (6379)
 - Elasticsearch (9200 , when search features are used)

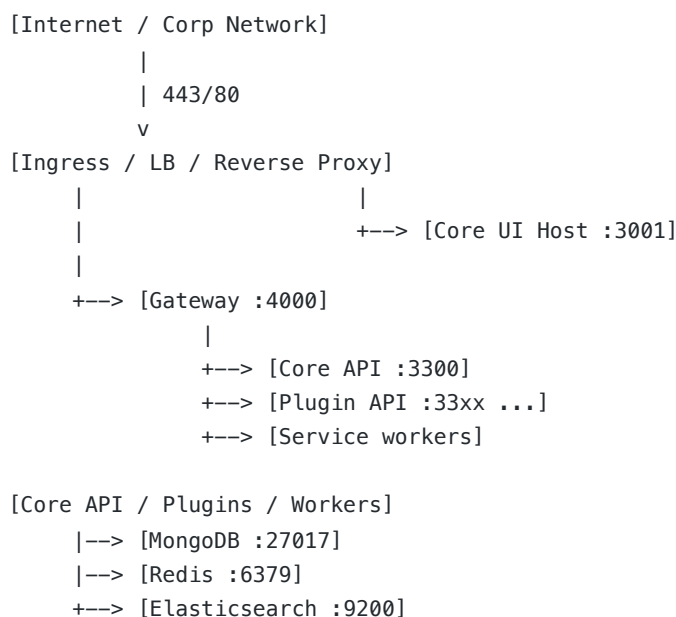
6.2 Containerized/Production Topology (Reference)

- Public ingress routes to:
 - `core-ui` (web)
 - `gateway` (API)
- `gateway` , `core-api` , plugin APIs, and services run as separate deployable units
- East-west traffic restricted to private network/subnets
- Shared infra services:
 - managed/self-hosted MongoDB
 - managed/self-hosted Redis
 - managed/self-hosted Elasticsearch

Recommended deployment pattern:

1. Edge proxy / ingress
2. Web tier (`core-ui` + plugin UI assets)
3. API tier (`gateway` + core + plugins + workers)
4. Data tier (MongoDB, Redis, Elasticsearch)

7) Network Topology



Security boundaries:

- Only ingress-exposed endpoints are public
- Core/plugin APIs should be private behind gateway in production
- Data services (MongoDB/Redis/Elasticsearch) remain private network-only

8) Runtime Dependency Map

8.1 Service Dependency Matrix

Service	Gateway	MongoDB	Redis	BullMQ	Elasticsearch	Notes
Core UI	Optional (direct API URL)	No	No	No	No	Loads plugin UIs via module federation
Plugin UI	Via Core UI/Gateway	No	No	No	No	Enabled from ENABLED_PLUGINS
Gateway	N/A	No	Yes	Indirect	No	Uses discovery/config records in Redis
Core API	Via Gateway	Yes	Yes	Optional	Optional	Core business modules
Plugin API	Via Gateway	Yes	Yes	Optional	Optional	Depends on plugin feature set
Automations Service	Via API ecosystem	Optional	Yes	Yes	Optional	Worker-oriented workload
Logs Service	Via API ecosystem	Optional	Yes	Optional	Optional	Logging/event handling path

8.2 Build/Repo Dependency Map

- Monorepo orchestrator: Nx (project.json graph)
- Package manager: pnpm
- Shared backend library: backend/erxes-api-shared
- Frontend shared libraries:
 - frontend/libs/erxes-ui
 - frontend/libs/ui-modules

9) Plugin Enablement and Configuration

Enablement is runtime-driven and environment-based.

- ENABLED_PLUGINS :
 - Enables both API and UI plugin paths
 - API naming: <plugin>_api
 - UI naming: <plugin>_ui
- ENABLED_PLUGINS_ONLY_API :
 - Enables API-only plugin services

- `ENABLED_SERVICES` :
 - Enables background services as `<name>--service`

Example:

```
ENABLED_PLUGINS=operation,sales,frontline
ENABLED_PLUGINS_ONLY_API=onefit
ENABLED_SERVICES=automations,logs
```

10) Operational Contracts

- Service registration contract:
 - APIs register address/config in Redis on boot
- Health and startup ordering:
 - Redis and MongoDB must be reachable before API full readiness
- Routing contract:
 - Clients should use gateway/API URL instead of calling plugin APIs directly

11) Living Doc Maintenance

Update this file when any of the following changes:

- New plugin/service added or retired
- Port, network route, or ingress changes
- Data store changes (new DB/cache/index)
- Queue/event model changes
- Core shared dependency changes (e.g., `erxes-api-shared` contracts)

Suggested review cadence:

- At every release cut
- After infrastructure changes
- During incident postmortems that reveal architecture drift

Change log template:

```
YYYY-MM-DD | owner/team | change summary | impacted sections
```